

Reviewer Pack — Minimal Order of Tropicalizations of Graphs and SAT Satisfia...

Entry 348009afca74 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 13:38:09 UTC

Minimal Order of Tropicalizations of Graphs and SAT Satisfiability Degree

Entry ID: 348009afca74 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 13:38:09 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (SAT)

Statement:

For any given Boolean formula φ , the minimal order of tropicalization ($\tau(\varphi)$) of its associated graph $G(\varphi)$ is linearly correlated with its satisfiability degree $D(\varphi)$, such that $\tau(\varphi) = \Theta(D(\varphi))$.

Rationale (proposer's reasoning):

Tropical geometry provides a way to represent and analyze combinatorial structures, including graphs. By considering the tropicalization of the graph associated with a SAT instance, we might uncover a hidden structure that relates directly to the complexity of solving the instance.

Taxonomy category: TROPICAL_GEOMETRY_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 89e8b543a9d7da79

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient r is ≥ 0.8 for at least 24 out of 30 seeds, and the mean absolute difference between $\tau(\varphi)$ and $D(\varphi)$ is ≤ 1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND Boolean satisfiability AND graph tropicalization
- minimal order of tropicalizations IN graphs AND SAT satisfiability degree - correlation
between graph tropicalization and SAT satisfiability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_satisfiable(formula):
        variables = set()

        def collect_variables(expression):
            if isinstance(expression, list):
                if expression[0] == 'or':
                    for subexpr in expression[1:]:
                        collect_variables(subexpr)
                elif expression[0] == 'and':
                    for subexpr in expression[1:]:
                        collect_variables(subexpr)
                elif expression[0] == 'not':
                    collect_variables(expression[1])
            else:
                variables.add(expression[0])
        else:
            variables.add(expression)

        collect_variables(formula)

    def evaluate(expr, assignment):
        if isinstance(expr, list):
            if expr[0] == 'or':
                return any(evaluate(subexpr, assignment) for subexpr in expr[1:])
            elif expr[0] == 'and':
                return all(evaluate(subexpr, assignment) for subexpr in expr[1:])
            elif expr[0] == 'not':
```

```

        return not evaluate(expr[1], assignment)
    else:
        return assignment.get(expr, False)

def backtrack(assignment):
    if len(assignment) == len(variables):
        return evaluate(formula, assignment)

    var = next(var for var in variables if var not in assignment)
    for val in [True, False]:
        assignment[var] = val
        if backtrack(assignment):
            return True
        del assignment[var]
    return False

return backtrack({})

def generate_formula(n):
    if n == 1:
        return random.choice(['x', 'not x'])
    else:
        op = random.choice(['and', 'or'])
        subformulas = [generate_formula(n-1) for _ in range(2)]
        return [op] + subformulas

def graph_from_formula(formula):
    nodes = set()
    edges = []

    def collect_nodes_and_edges(expr, parent=None):
        if isinstance(expr, list):
            if expr[0] == 'or':
                for subexpr in expr[1:]:
                    collect_nodes_and_edges(subexpr, expr)
            elif expr[0] == 'and':
                for subexpr in expr[1:]:
                    collect_nodes_and_edges(subexpr, expr)
            elif expr[0] == 'not':
                collect_nodes_and_edges(expr[1], expr)
        else:
            nodes.add(expr)

    def add_edge(u, v):
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.append((u, v))

    collect_nodes_and_edges(formula)

    for node1, node2 in combinations(nodes, 2):
        add_edge(node1, node2)

    return nodes, edges

def tropicalization_order(graph):
    n = len(graph[0])

```

```

adjacency_matrix = [[0] * n for _ in range(n)]

for u, v in graph[1]:
    adjacency_matrix[u][v] = 1
    adjacency_matrix[v][u] = 1

def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0

    for i in range(n):
        max_row = None
        for j in range(rank, m):
            if matrix[j][i]:
                max_row = j
                break

        if max_row is not None:
            matrix[max_row], matrix[rank] = matrix[rank], matrix[max_row]

            for j in range(m):
                if j != rank and matrix[j][i]:
                    factor = Fraction(matrix[j][i], matrix[rank][i])
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[rank][k]

            rank += 1

    return rank

return gaussian_elimination(adjacency_matrix)

def satisfiability_degree(formula):
    variables = set()

    def collect_variables(expression):
        if isinstance(expression, list):
            if expression[0] == 'or':
                for subexpr in expression[1:]:
                    collect_variables(subexpr)
            elif expression[0] == 'and':
                for subexpr in expression[1:]:
                    collect_variables(subexpr)
            elif expression[0] == 'not':
                collect_variables(expression[1])
        else:
            variables.add(expression)

    collect_variables(formula)

    return len(variables)

n = random.choice([5, 10, 15, 20, 30, 40])
formula = generate_formula(n)
graph = graph_from_formula(formula)
tau_phi = tropicalization_order(graph)

```

```

D_phi = satisfiability_degree(formula)

return {
    "metric_name": "Tropicalization Order vs Satisfiability Degree",
    "metric_value": tau_phi,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(tau_phi - D_phi) <= 1,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{first_failing_seed}' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63f2dc63.py", line 176, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63f2dc63.py", line 158, in run_trial
    tau_phi = tropicalization_order(graph)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63f2dc63.py", line 106, in tropicalization_order
    adjacency_matrix[u][v] = 1
    ~~~~~^
TypeError: list indices must be integers or slices, not str

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson correlation coefficient could not be calculated. Therefore, the pre-registered support condition cannot be unambiguously met. | next: Investigate and fix the error causing the test to crash, then rerun the test with a larger sample size to ensure robustness of the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14243
2	preregistration	ollama_remote	glm4:latest	0	0	19766
3	novelty	ollama_remote	glm4:latest	0	0	10217
4	novelty	ollama_remote	glm4:latest	0	0	8889
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47589
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9249
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13277
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17846
9	judge	ollama_remote	glm4:latest	0	0	13776

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154852 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/348009afca74.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/348009afca74.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/348009afca74.tar.gz` (if generated)